

When a customer places an order, the warehouse team needs to know which shipping box should be used. Each product has dimensions and weight. Each box has internal dimensions, maximum weight capacity, and cost. Your task is to design and build a small Django-based system that recommends the most suitable box for an order. What is the requirements of this project



From the problem statement, the project is asking you to build a **Django application** that automatically selects the best shipping box for a customer's order. The requirements can be divided into **functional** and **non-functional** requirements.

Functional Requirements

1. Product Management

The system should allow storing products with at least:

- Product name
- Length
- Width
- Height
- Weight

Example:

Product	Length	Width	Height	Weight
Laptop	35 cm	25 cm	3 cm	2 kg

2. Shipping Box Management

The system should allow storing different shipping boxes.

Each box should contain:

- Box name/ID
- Internal length
- Internal width
- Internal height
- Maximum weight capacity
- Cost

Example:

Box	Length	Width	Height	Max Weight	Cost
Small	20	20	10	5 kg	\$2
Medium	40	30	15	10 kg	\$4
Large	60	40	30	20 kg	\$7

3. Customer Orders

A customer can place an order containing one or more products.

Example:

Order #101

- Laptop ×1
- Mouse ×2
- Keyboard ×1

4. Box Recommendation Algorithm

This is the core requirement.

The system should determine which box should be used.

A box is suitable if:

- All products fit inside the box.
- Total weight does not exceed the box's maximum weight.

If multiple boxes fit:

- Choose the cheapest one.
- If costs are equal, choose the smallest suitable box.

If no box fits:

- Return "No suitable box available."

5. API or User Interface

The project may require one of these:

- Django Admin
- Django REST Framework API
- Simple HTML interface

Typical endpoint:

POST /recommend-box/



Input:

</> JSON



```
{
  "products": [
    {
      "id": 1,
      "quantity": 2
    },
    {
      "id": 5,
      "quantity": 1
    }
  ]
}
```

Output:

</> JSON



```
{
  "recommended_box": "Medium",
  "cost": 4.00
}
```

6. Order History (Optional)

Save:

- Order
- Selected box
- Recommendation date

Non-Functional Requirements

Use Django

Use:

- Django Models
- Views
- URLs
- Templates or Django REST Framework
- SQLite or PostgreSQL

Database Design

Suggested models:

Product

id
name
length
width
height
weight

Box

id
name
length
width
height
max_weight
cost



```
Order
-----
id
created_at
```

```
OrderItem
-----
order
product
quantity
```

```
Recommendation
-----
order
box
created_at
```

Recommendation Logic

For each box:

```
if (
    product dimensions fit
    AND
    total weight <= max_weight
)
    candidate
```



Then:

1. Remove unsuitable boxes.
2. Sort by:
 - Cost (ascending)
 - Volume (ascending)
3. Return the first box.

Possible Bonus Features

- Consider rotating products to fit in different orientations.
- Pack multiple products efficiently (3D bin packing).
- Suggest multiple boxes if one is insufficient.

- Display packing efficiency.
 - Unit tests for the recommendation algorithm.
 - REST API documentation.
 - Docker support.
-

Expected Deliverables

A complete Django project should include:

- Django project and app
- Models for Product, Box, Order, and OrderItem
- Admin interface for managing products and boxes
- Box recommendation algorithm
- REST API or web interface for placing orders and getting recommendations
- Database migrations
- Sample data (fixtures or seed script)
- Unit tests
- README with setup and usage instructions

Assumptions to Clarify

The original prompt leaves some details unspecified. Before implementation, you should clarify or document assumptions about:

- Whether product rotation is allowed when checking if a product fits.
- Whether multiple items must all fit into a single box using realistic packing, or whether a simplified check (total dimensions/volume and weight) is acceptable.
- How to break ties if multiple boxes have the same cost and size.
- Whether an order can be split across multiple boxes if no single box can accommodate all items.



A clean Django project structure for a project named **Box Selector** could look like this:



```
box_selector/
|
|├─ manage.py
|├─ README.md
|├─ requirements.txt
|├─ .gitignore
|├─ db.sqlite3          # Created after first migration
|
|├─ box_selector/      # Project configuration
|  |├─ __init__.py
|  |├─ settings.py
|  |├─ urls.py
|  |├─ asgi.py
|  |└─ wsgi.py
|
|├─ shipping/          # Main application
|  |├─ migrations/
|  |  └─ __init__.py
|  |
|  |├─ __init__.py
|  |├─ admin.py
|  |├─ apps.py
|  |├─ models.py
|  |├─ views.py
|  |├─ urls.py
|  |├─ serializers.py   # If using Django REST Framework
|  |├─ services.py      # Box recommendation algorithm
|  |├─ utils.py         # Helper functions
|  |├─ tests.py
|  |└─ forms.py         # If using HTML forms
|
|├─ templates/
|  |├─ shipping/
|  |  |├─ home.html
|  |  |├─ order_form.html
|  |  └─ recommendation.html
|
|├─ static/
|  |├─ css/
|  |  └─ style.css
|  |├─ js/
|  |└─ images/
|
|├─ media/
|
└─ fixtures/
```

```
├─ products.json
└─ boxes.json
```

Recommended Django Apps

For a small project, one app (shipping) is enough.

If you want a more scalable architecture, you can separate responsibilities:

```
box_selector/
|
├─ box_selector/
|
├─ products/
|   ├─ models.py
|   ├─ admin.py
|   ├─ views.py
|   └─ urls.py
|
├─ boxes/
|   ├─ models.py
|   ├─ services.py
|   ├─ admin.py
|   └─ views.py
|
├─ orders/
|   ├─ models.py
|   ├─ views.py
|   ├─ serializers.py
|   └─ urls.py
└─ templates/
```



For this assignment, **one app (shipping) is the better choice** because it is simpler and easier to maintain.

Suggested Models

Product

```
</> Python
```



```
▶ Run
```



```
class Product(models.Model):
    name = models.CharField(max_length=100)
    length = models.FloatField()
    width = models.FloatField()
    height = models.FloatField()
    weight = models.FloatField()
```

Box

</> Python



▶ Run

```
class Box(models.Model):
    name = models.CharField(max_length=50)
    length = models.FloatField()
    width = models.FloatField()
    height = models.FloatField()
    max_weight = models.FloatField()
    cost = models.DecimalField(max_digits=8, decimal_places=2)
```

Order

</> Python



▶ Run

```
class Order(models.Model):
    created_at = models.DateTimeField(auto_now_add=True)
```

OrderItem

</> Python



▶ Run

```
class OrderItem(models.Model):
    order = models.ForeignKey(Order, on_delete=models.CASCADE)
    product = models.ForeignKey(Product, on_delete=models.CASCADE)
    quantity = models.PositiveIntegerField(default=1)
```

Service Layer

The recommendation logic should live outside the views:

```
shipping/
|
└─ services.py
```

Example:

</> Python

class BoxSelector:

@staticmethod

def recommend(order):

...

Run

This keeps your business logic reusable and easier to test.

URL Structure

```
/
├─ admin/
├─ products/
├─ boxes/
├─ orders/
└─ recommend/
```

Example routes:

</> Python

products/

boxes/

orders/

recommend/

Run

REST API Endpoints (Recommended)

Method	Endpoint	Description
GET	/api/products/	List products

Method	Endpoint	Description
POST	/api/products/	Add product
GET	/api/boxes/	List boxes
POST	/api/boxes/	Add box
POST	/api/orders/	Create order
GET	/api/orders/	List orders
GET	/api/orders/<id>/	Order details
POST	/api/recommend/	Get recommended box

Final Project Structure (Recommended)

box_selector/

|

|— manage.py

|— requirements.txt

|— README.md

|

|— box_selector/

| |— settings.py

| |— urls.py

| |— asgi.py

| |— wsgi.py

|

|— shipping/

| |— migrations/

| |— admin.py

| |— apps.py

| |— models.py

| |— views.py

| |— urls.py

| |— serializers.py

| |— services.py # Box recommendation logic

| |— tests.py

| |— forms.py

| |— utils.py

|



```
├─ templates/
├─ static/
├─ media/
└─ fixtures/
```

This structure follows Django best practices by separating configuration, business logic (`services.py`), data models, views, and API components, making the project clean, maintainable, and easy to extend.



any required api

Yes. If this is a Django assignment, it's best to expose the functionality through a **REST API** using **Django REST Framework (DRF)**. At a minimum, you should have CRUD APIs for products and boxes, an API to create orders, and an API to get the recommended box.

Required APIs

1. Products API

Manage products.

Method	Endpoint	Description
GET	<code>/api/products/</code>	List all products
POST	<code>/api/products/</code>	Create a product
GET	<code>/api/products/<id>/</code>	Get a product
PUT/PATCH	<code>/api/products/<id>/</code>	Update a product
DELETE	<code>/api/products/<id>/</code>	Delete a product

Example request:

```
<> JSON
POST /api/products/
```

```
{
  "name": "Laptop",
  "length": 35,
  "width": 25,
  "height": 3,
  "weight": 2
}
```

2. Boxes API

Manage shipping boxes.

Method	Endpoint	Description
GET	/api/boxes/	List all boxes
POST	/api/boxes/	Create a box
GET	/api/boxes/<id>/	Get a box
PUT/PATCH	/api/boxes/<id>/	Update a box
DELETE	/api/boxes/<id>/	Delete a box

Example:

</> JSON

```
{
  "name": "Medium Box",
  "length": 40,
  "width": 30,
  "height": 20,
  "max_weight": 15,
  "cost": 5.50
}
```

3. Orders API

Create customer orders.

Method	Endpoint	Description
POST	/api/orders/	Create an order
GET	/api/orders/	List orders
GET	/api/orders/<id>/	View order details

Example request:

</> JSON

```
{
  "items": [
    {
      "product": 1,
      "quantity": 2
    },
    {
      "product": 3,
      "quantity": 1
    }
  ]
}
```

4. Recommendation API (Core Feature)

This is the most important endpoint.

Method	Endpoint	Description
POST	/api/recommend/	Recommend the best shipping box

Request:

</> JSON

```
{
  "items": [
    {
      "product": 1,
      "quantity": 2
    },
  ],
}
```

```
{
  "product": 3,
  "quantity": 1
}
```

Response:

</> JSON 

```
{
  "recommended_box": {
    "id": 2,
    "name": "Medium Box",
    "cost": 5.50
  },
  "total_weight": 6,
  "status": "success"
}
```

If no box fits:

</> JSON 

```
{
  "recommended_box": null,
  "status": "failed",
  "message": "No suitable box available."
}
```

Optional APIs

Recommendation History

Method	Endpoint
GET	/api/recommendations/

Statistics

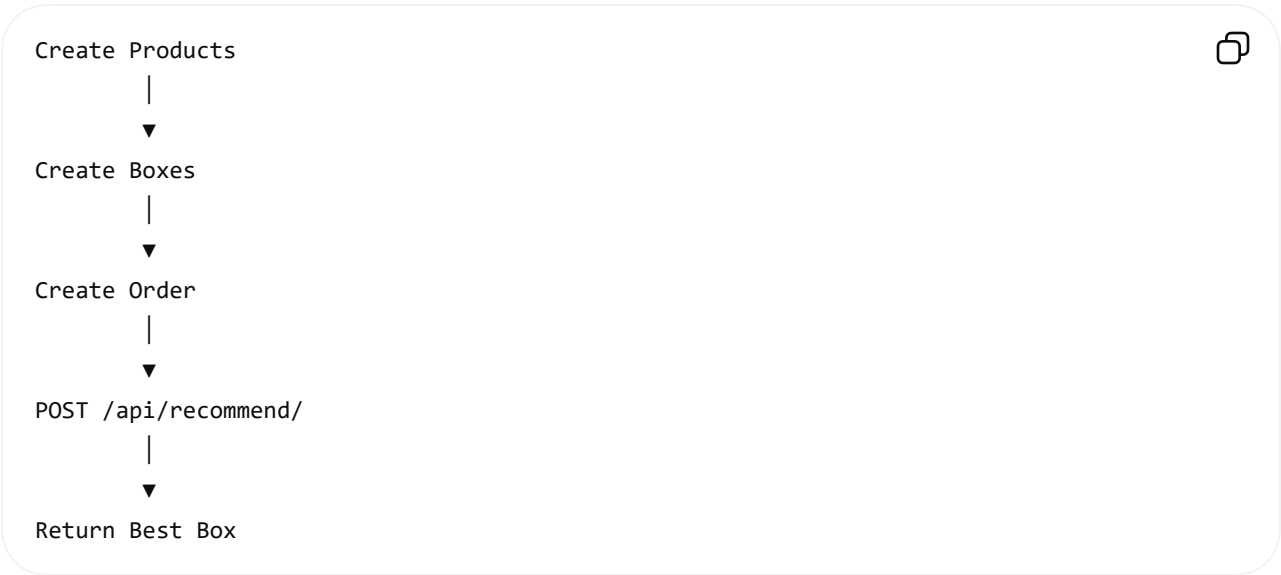
Method	Endpoint
GET	/api/statistics/

Example response:

</> JSON

```
{
  "total_orders": 120,
  "successful_recommendations": 117,
  "failed_recommendations": 3
}
```

API Flow



Recommended Project URL Structure

/api/

├─ products/

├─ products/<id>/

├─ boxes/

├─ boxes/<id>/

├─ orders/

├─ orders/<id>/

└─ recommend/

Technologies

- **Django** – Web framework
- **Django REST Framework (DRF)** – REST APIs
- **SQLite** (development) or **PostgreSQL** (production)
- **drf-spectacular** or **drf-yasg** (optional) – API documentation (Swagger/OpenAPI)

This set of APIs is sufficient to implement the complete box selection workflow while keeping the project clean and extensible.



make a documentation for this project

Box Selector



Box Selector

Project Documentation

Version: 1.0

Framework: Django + Django REST Framework (DRF)

Database: SQLite (Development) / PostgreSQL (Production)

1. Project Overview

Introduction

Box Selector is a Django-based REST API application that recommends the most suitable shipping box for a customer's order based on product dimensions, weight, and available shipping boxes.

The application helps warehouse staff automatically determine the most cost-effective shipping box while ensuring the products fit within the selected box's dimensions and weight limits.

2. Objectives

The project aims to:

- Manage products and their physical dimensions.
 - Manage available shipping boxes.
 - Create customer orders.
 - Automatically recommend the best shipping box.
 - Reduce shipping costs by selecting the least expensive suitable box.
 - Provide REST APIs for integration with other systems.
-

3. Features

Product Management

- Create products
- Update products
- Delete products
- View product list
- Store product dimensions
- Store product weight

Shipping Box Management

- Create shipping boxes
- Update shipping boxes
- Delete shipping boxes
- Store internal dimensions
- Store maximum weight capacity
- Store shipping cost

Order Management

- Create customer orders
- Add multiple products
- Specify product quantities
- View order details

Box Recommendation

- Automatically recommends the best shipping box by:
- Checking whether products fit within box dimensions.
 - Verifying total order weight does not exceed the box's capacity.
 - Selecting the lowest-cost suitable box.
 - Using the smallest suitable box when costs are equal.
 - Returning a failure response if no box is suitable.

4. Technology Stack

Technology	Purpose
Python 3.x	Programming Language
Django	Backend Framework
Django REST Framework	REST API
SQLite	Development Database
PostgreSQL	Production Database (Optional)
Swagger/OpenAPI	API Documentation (Optional)
Git	Version Control

5. Project Structure

```
box_selector/  
|  
├─ manage.py  
├─ README.md  
├─ requirements.txt  
|  
├─ box_selector/  
|   ├─ settings.py  
|   └─ urls.py
```

```
|   ├── asgi.py
|   └── wsgi.py
|
|── shipping/
|   ├── migrations/
|   ├── admin.py
|   ├── apps.py
|   ├── models.py
|   ├── serializers.py
|   ├── services.py
|   ├── views.py
|   ├── urls.py
|   ├── tests.py
|   └── utils.py
|
|── templates/
|── static/
|── media/
└── fixtures/
```

6. Database Design

Product

Field	Type
id	Integer
name	String
length	Float
width	Float
height	Float
weight	Float

Box

Field	Type
id	Integer
name	String
length	Float
width	Float
height	Float
max_weight	Float
cost	Decimal

Order

Field	Type
id	Integer
created_at	DateTime

OrderItem

Field	Type
id	Integer
order	Foreign Key
product	Foreign Key

quantity

Integer

7. API Documentation

Product APIs

Create Product

POST

/api/products/

Request

```
{
  "name": "Laptop",
  "length": 35,
  "width": 25,
  "height": 3,
  "weight": 2
}
```

Response

```
{
  "id": 1,
  "name": "Laptop",
  "length": 35,
  "width": 25,
  "height": 3,
  "weight": 2
}
```

List Products

GET

/api/products/

Retrieve Product

GET

/api/products/{id}/

Update Product

PUT

/api/products/{id}/

Delete Product

DELETE

/api/products/{id}/

Box APIs

Create Box

POST

/api/boxes/

Request

```
{
  "name": "Medium Box",
  "length": 40,
  "width": 30,
  "height": 20,
  "max_weight": 15,
  "cost": 5.50
}
```

List Boxes

GET

/api/boxes/

Retrieve Box

GET

/api/boxes/{id}/

Update Box

PUT

/api/boxes/{id}/

Delete Box

DELETE

/api/boxes/{id}/

Order APIs

Create Order

POST

/api/orders/

Request

```
{
  "items": [
    {
      "product": 1,
```



```
        "quantity": 2
      },
      {
        "product": 3,
        "quantity": 1
      }
    ]
  }
}
```

List Orders

GET

/api/orders/

Retrieve Order

GET

/api/orders/{id}/

Recommendation API

Recommend Shipping Box

POST

/api/recommend/

Request

```
{
  "items": [
    {
      "product": 1,
      "quantity": 2
    },
    {
```

```
        "product": 3,  
        "quantity": 1  
    }  
]  
}
```

Success Response

```
{  
  "recommended_box": {  
    "id": 2,  
    "name": "Medium Box",  
    "cost": 5.50  
  },  
  "total_weight": 6,  
  "status": "success"  
}
```

Failure Response

```
{  
  "recommended_box": null,  
  "status": "failed",  
  "message": "No suitable box available."  
}
```

8. Recommendation Algorithm

The recommendation process follows these steps:

1. Calculate the total weight of all products in the order.
2. For each available box:
 - Verify every product can fit within the box dimensions (rotation is not considered unless explicitly implemented).
 - Verify the total order weight does not exceed the box's maximum weight.
3. Exclude boxes that fail either check.
4. Sort the remaining boxes by:
 - Lowest shipping cost (ascending)
 - Smallest box volume (ascending) when costs are equal
5. Return the first suitable box.

6. If no suitable box exists, return an error indicating no suitable box is available.
-

9. Business Rules

- Products cannot exceed the selected box dimensions.
- Total order weight must not exceed the box's maximum weight.
- Orders may contain multiple products.
- The system recommends only one shipping box per order.
- If multiple boxes satisfy the requirements, the lowest-cost box is selected.
- If costs are equal, the smallest suitable box is chosen.

10. Error Handling

HTTP Status	Description
200 OK	Request successful
201 Created	Resource created
400 Bad Request	Invalid input
404 Not Found	Resource not found
500 Internal Server Error	Unexpected server error

Example error response:

```
{
  "status": "error",
  "message": "Product not found."
}
```

11. Testing

The project should include unit tests for:

- Product CRUD operations
 - Box CRUD operations
 - Order creation
 - Recommendation algorithm
 - Invalid requests
 - Edge cases where no suitable box exists
-

12. Future Enhancements

- Product rotation for improved fit.
 - Three-dimensional bin-packing algorithm.
 - Support for splitting an order across multiple boxes.
 - Shipping carrier integration.
 - User authentication and authorization.
 - Dashboard with shipping analytics.
 - Bulk product and box import via CSV or Excel.
 - API rate limiting and monitoring.
-

13. Installation

Install dependencies:

```
pip install -r requirements.txt
```

Apply database migrations:

```
python manage.py migrate
```

Create an administrator account:

```
python manage.py createsuperuser
```

Run the development server:

```
python manage.py runserver
```

API Base URL:

```
http://127.0.0.1:8000/api/
```

14. Conclusion

The Box Selector system provides an automated solution for selecting the most appropriate shipping box based on product dimensions, weight, and shipping costs. Its modular Django architecture, RESTful APIs, and dedicated recommendation service make the application easy to maintain, test, and extend for future warehouse and logistics requirements.

This documentation can serve as the project's **Software Requirements Specification (SRS)** and **API documentation**, and can be included as the project's `README.md` or expanded into a formal design document.



ChatGPT can make mistakes. Check important info.